

文件操作

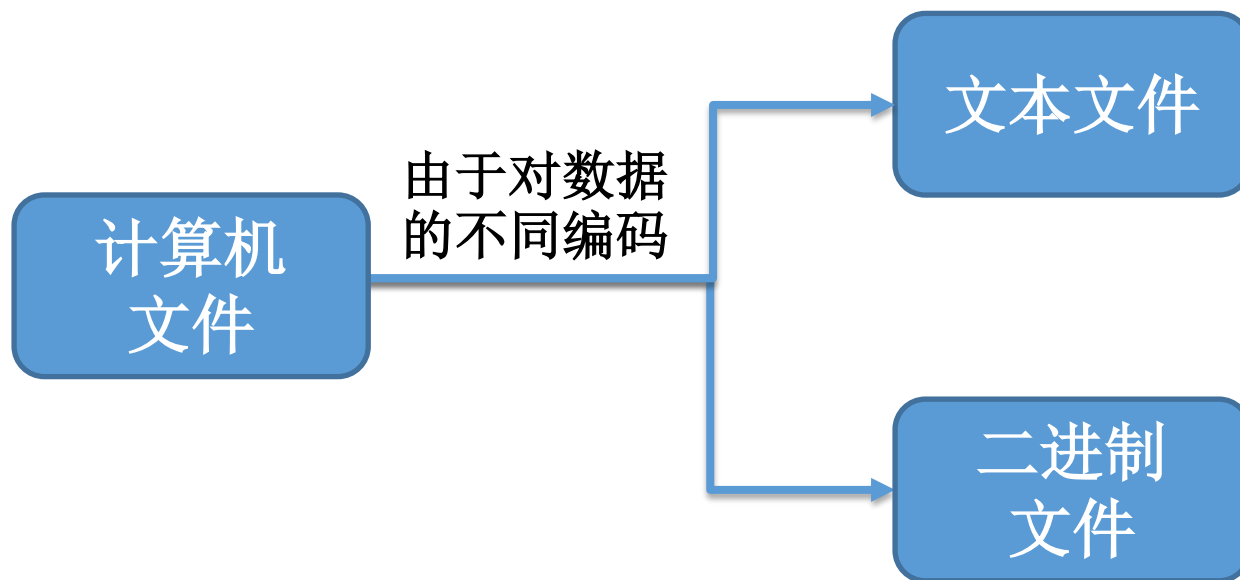


- ◆ 掌握文件的基本操作
- ◆ 掌握二进制文件的处理
- ◆ 了解文件的系统操作
- ◆ 了解常见文件格式的读写

按文件中数据的组织形式把文件分为**文本文件**和**二进制文件**两类。

文本文件：存储的是常规字符串，由若干文本行组成，每行以换行符“**\n**”结尾。可以使用文本编辑器进行**显示、编辑并且能够直接阅读**。如网页文件、记事本文件、程序源代码文件等。

二进制文件：存储的是字节串**bytes**，需要使用专门的软件进行解码后才能读取、显示、修改或执行。如图形图像文件、音视频文件、可执行文件、数据库文件等。



文本文件可以用普通文本编辑器直接查看，通常是以字符串形式存在的字母、汉字和数字等。

二进制文件则呈现了各种形态，包括字处理文件、可执行文件、图像文件、音视频文件等。

- 向（从）一个文件写（读）数据之前，需要**先创建一个和物理文件相关的文件对象**，然后**通过该文件对象**对文件内容进行**读取、写入、删除、修改**等操作，最后**关闭并保存**文件内容。
- Python内置的**open()**函数可以按指定的模式打开指定的文件并创建文件对象。打开一个文件并创建文件对象的语法如下。

file_object = open(file, mode='r', buffering=-1, encoding)

open函数打开文件**file**，返回一个指向文件**file**的文件对象**file_object**。

➤ **`file_object = open(file, mode='r', buffering=-1, encoding)`**

参数的说明如下：

filename: 要访问的文件名称。 **file**是一个包含文件所在路径及文件名称的字符串值，如'`c:\\User\\test.txt`'。

mode: 指定打开文件后的处理方式：只读，写入，追加等。这个参数是非强制的，默认文件访问模式为只读(`r`)。

buffering: 0表示不缓存，1表示缓存。大于1表示缓冲区的大小。-1表示缓冲区的大小为系统默认值。

encoding: 指定对文本进行编码和解码的方式，只适用于文本模式，可以使用Python支持的任何格式，如`gbk`、`utf8`、`cp936`等。

➤ Python使用`open()`方法打开一个文件，并返回一个可迭代的文件对象，通过该文件对象可以对文件进行读写操作。如果文件不存在、访问权限不够、磁盘空间不足或其它原因导致创建文件对象失败，`open()`函数就会抛出一个`IOError`的错误，并且给出错误码和详细的信息。

- 磁盘满、无法写入，打开文件要读取但文件不存在，或者文件路径错误。这些是文件读写过程中随时可能遇到的问题。
- 凡是涉及文件输入输出的操作，这类问题在程序设计时是必然要考虑的因素，否则程序的设计并不完整和严谨。

➤ 文件打开模式

打开模式定义了打开文件后的处理方式，如只读、读写、追加等。

模 式	功能描述
r	只读模式
w	写模式，若存在则覆盖原有内容
a	追加模式
r+	读写，不创建
w+	读写，若不存在则新建
a+	追加模式但可读
rt, wt	默认为文本方式，相当于 r, w
rb, wb	读写二进制文件

文件打开的不同模式：

模式	描述
r	以只读方式打开文件，文件的指针放在文件的开头。这是默认模式，可省略。
r+	以读写格式打开一个文件，文件指针放在文件的开头。
w	以写入格式打开一个文件，如果该文件已存在，则将其覆盖。如果该文件不存在，则创建新文件。
w+	以读写格式打开一个文件，如果该文件已存在则将其覆盖。如果该文件不存在，则创建新文件。
a	以追加格式打开一个文件，如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
a+	以读写格式打开一个文件，如果该文件已存在，文件指针将会放在文件的结尾。如果该文件不存在，创建新文件用于读写。

● 打开文件

一个文件被打开后，返回一个文件对象，通过文件对象可以得到有关该文件的各种信息。文件对象的属性如表所示：

属性	描述
closed	判断文件是否关闭，如果文件已被关闭，返回 True ；否则返回 False
mode	返回被打开文件的访问模式
name	返回文件的名称

一个文件被打开后，返回一个文件对象，通过文件对象可以得到有关该文件的各种信息。文件对象的属性举例如下：

```
In [29]: ► file_object=open('c:\\test\\scores.txt', 'r')
          print('文件名：', file_object.name)
          print('是否已关闭：', file_object.closed)
          print('访问模式：', file_object.mode)
```

```
文件名： c:\test\scores.txt
是否已关闭： False
访问模式： r
```

文件对象的常用方法如表所示：

方法	功能说明
close()	刷新缓冲区里还没写入的信息，并关闭该文件
flush()	刷新文件内部缓冲，把内部缓冲区的数据立刻写入文件，但不关闭文件
next()	返回文件下一行
read([size])	从文件的开始位置读取指定的size个字符数，如果未给定则读取所有
readline()	读取整行，包括"\n"字符
readlines()	把文本文件中的每行文本作为一个字符串存入列表中并返回该列表

文件对象的常用方法如表所示：

方法	功能说明
seek(offset[,from])	用于移动文件读取指针到指定位置， offset 为需要移动的字节数； from 指定从哪个位置开始移动，默认值为0，0代表从文件开头开始，1代表从当前位置开始，2代表从文件末尾开始。
tell()	方法返回文件的当前位置，即文件指针当前位置
truncate([size])	删除从当前指针位置到文件末尾的内容。如果指定了 size ，则不论指针在什么位置都只留下前 size 个字符，其余的删除
write(str)	把字符串 str 的内容写入文件中，没有返回值。由于缓冲，字符串内容可能没有加入到实际的文件中，直到调用 flush() 或 close() 方法被调用。
writelines([str])	用于向文件中写入字符串序列
writable()	测试当前文件是否可写
readable()	测试当前文件是否可读

文件的关闭:

- 当对文件内容操作完成以后，必须关闭文件，这样能够保证所做的修改得到保存。
- `close()` 方法用于关闭一个已打开的文件，可以将缓冲的数据写入文件，然后关闭文件。关闭后的文件不能再进行读写操作，否则会触发 `ValueError` 错误。`close()` 方法允许调用多次。
- 而 `flush()` 方法将缓冲的数据写入文件，但是不关闭文件。

● 文件对象

➤ 使用上下文管理语句**with**可以**自动管理资源**，在代码块执行完毕后**自动还原**进入该代码块之前的**现场或上下文**。

● **with**语句的语法如下：

with context_expr [as var]:

with语句块

```
with open("myfile.txt") as f:  
    for line in f:  
        print(line, end="")
```


文件的关闭：

➤ 使用with语句的好处：

- 使用with自动关闭资源，可以在代码块执行完毕后还原进入该代码块时的现场。
- 不论何种原因跳出with块，不论是否发生异常，总能保证文件被正确关闭，资源被正确释放。

写文件：

当一个文件以“写”的方式打开后，可以使用 `write()` 方法和 `writelines()` 方法，将字符串写入文本文件。

➤ 1. `write()` 方法：

`write(s)` 方法用于向一个打开的文件中写入指定的字符串。在文件关闭前或缓冲区刷新前，字符串内容存储在缓冲区中，这时在文件中是看不到写入的内容的。需要重点注意的是，`write()` 方法不会在字符串的结尾添加换行符“`\n`”。

`write()` 方法语法格式如下：`fileObject.write(str)`

参数 `str`：要写入文件的字符串。

返回值：返回的是写入的字符长度。

在操作文件时，每调用一次 `write()` 方法，写入的数据就会追加到文件末尾。

写文件：

1. write() 方法

➤程序运行后，会在程序当前的路径下，生成一个名为 test.txt 文件，打开该文件，可以看到数据成功被写入。

➤注意：当向文件写入数据时，如果文件不存在，那么系统会自动创建一个文件并写入数据。如果文件存在，那么会清空原来文件的数据，重新写入新数据。

```
In [35]:  ► fp= open("test.txt", "w") #以只写方式打开文本文件
          fp.write("My name is Guido van Rossum!\n")
          fp.write("I invented the Python programming language!\n")
          fp.write("I love Python!\n")
          fp.close()
```

写文件：

2. writelines() 方法

➤ writelines() 方法把字符串列表写入文本文件，**不添加换行符 “\n”**。

```
In [43]: ► data = ['My ', 'name ', 'is ', 'John!\n', 'How ', 'are ', 'you!'] #生成要写入的列表内容
with open("data_desc.txt", "w") as fp:
    fp.writelines(data)
with open("data_desc.txt", "r") as fp:
    for line in fp:
        print(line)
```

My name is John!

How are you!

读文件：

当一个文件被打开后，可使用三种方式从文件中读取数据：`read()`、`readline()`、`readlines()`。

➤ `read([size])`：从文件读取指定的`size`个字符数，如果未给定则读取所有。

➤ `readline()`：读取整行。

➤ `readlines()`：把文本文件中的每行文本作为一个字符串存入列表中并返回该列表。

注意：打开一个已经存在的文件，读取正常，当试图打开一个不存在的文件，系统会给出错误信息。

读文件：

Python文件对象提供了三个“读”方法：`read()`、`readline()`和 `readlines()`。每种方法可以接受一个变量以限制每次读取的数据数量。

1. `read()` 方法

`read()` 方法从文件当前位置起读取size个字符串，若无参数size，则表示读取至文件结束为止。如果使用多次，那么后面读取的数据是从上次读完后的位置开始。

`read()` 语法结构如下：`fileObject.read(size)`

参数size：从文件中读取的字符数。如果没有指定字符数，那么就表示读取文件的全部内容。返回值：返回从字符串中读取的字符内容。

读文件：

1. `read()` 方法

```
In [44]: ▶ fp = open("test.txt", "r")  
          content = fp.read(10)  
          print(content)
```

#只读方式打开文本文件
#读取10个字节数据

My name is

读文件：

2. `readline()` 方法

该方法每次读出一行内容，该方法读取时占用内存小，比较适合大文件，该方法返回一个字符串对象。

`readline()` 语法：`fileObject.readline()`

返回值：返回读取的字符串。

读文件：readline() 方法案例：

```
fp= open("test1.txt", "r")    #只读方式打开文件
line = fp.readline()
print("读取第一行:%s" % (line))
print("-----华丽的分割线-----")
while line:                  #循环读取每一行
    print(line)
    line = fp.readline()
fp.close()
print("文件", fp.name, "已经成功分行读出！")
```

读取第一行:My name is Guido van Rossum!

-----华丽的分割线-----

My name is Guido van Rossum!

I invented the Python programming language!

I love Python!

文件 test1.txt 已经成功分行读出！

读文件：

3. `readlines()` 方法

`readlines()` 方法读取文件的所有行，保存在一个列表中，每行作为一个元素，但读取大文件会比较占内存。该列表可以由 Python 的 `for... in ...` 结构进行处理。

`readlines()` 语法：`fileObject.readlines()`

返回值：此方法返回包含所有行的列表。

读文件：readlines() 方法案例：

```
In [46]: ► fp=open("test.txt", "r")    #只读方式打开文件
lines = fp.readlines()
print("行的数据类型:", type(lines))
print(("列表形式存放每一行: %s" %(lines)))
print("-----分割线-----")
for line in lines: #依次读取每行
    line = line.strip()    #去掉每行头尾空白符, 包括'\n'
    print("读取的数据为: %s" % (line))
fp.close()
print("文件", f.name, "已经成功把所有行读出!")
```

行的数据类型: <class 'list'>

列表形式存放每一行: ['My name is Guido van Rossum!\n', 'I invented the Python programming language!\n', 'I love Python!\n']

-----分割线-----

读取的数据为: My name is Guido van Rossum!

读取的数据为: I invented the Python programming language!

读取的数据为: I love Python!

文件 test.txt 已经成功把所有行读出!

读文件：用`readlines()`方法读取一段中文诗词并打印。

➤ Windows平台下`open()`函数在打开文件的时候缺省的编码（encoding）为gbk(cp936)，并不是UTF-8，因此在打开文件的时候应指定编码为UTF-8，否则读取文件会出现错误。

```
In [48]: ▶ fname = r'C:\test\shi.txt'
with open(fname, 'r', encoding='utf-8') as f:
    lines = f.readlines()
    for line in lines: print(line.rstrip())
```

日照香炉生紫烟
遥看瀑布挂前川
飞流直下三千尺
疑是银河落九天

读文件：

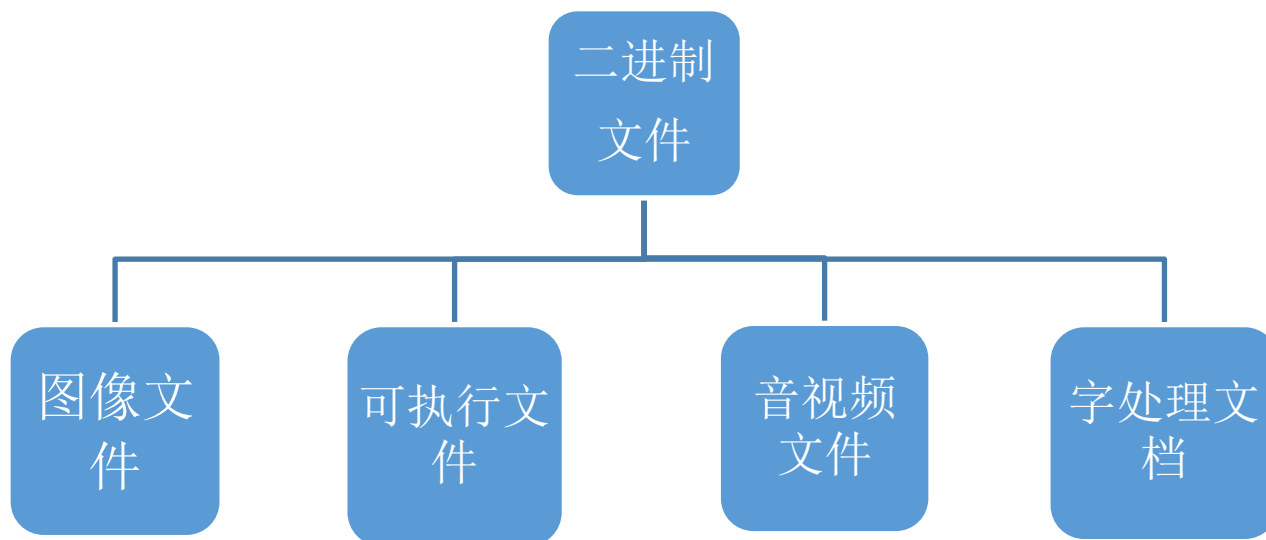
4. 从文件逐行读取数据：

```
In [47]: ► with open('test.txt') as f:  
           for line in f:  
               print(line, end='')
```

```
My name is Guido van Rossum!  
I invented the Python programming language!  
I love Python!
```

➤ 二进制文件

二进制文件包括图像文件、可执行文件、音视频文件、字处理文档等，**不能使用记事本或其他文本编辑软件直接读写。**



➤ 二进制文件的读写

二进制文件读写的是bytes字节串。运行下面程序，由于写入的是一个字符串，非字节串，系统会抛出异常。

```
In [23]: ► with open("test.bt", "wb") as fp:  
          fp.write("abcd")
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-23-1097f6556c11> in <module>  
      1 with open("test.bt", "wb") as fp:  
----> 2     fp.write("abcd")  
  
TypeError: a bytes-like object is required, not 'str'
```

➤ 读写二进制文件

表 2-7-2 二进制文件模式

文件使用方式	意 义
rb	只读打开一个二进制文件，只允许读数据。如文件存在，则打开后可以顺序读；如文件不存在，则打开失败。
wb	只写打开或建立一个二进制文件，只允许写数据。如文件不存在，则建立一个空文件；如文件已经存在，则把原文件内容清空。
ab	追加打开一个文本文件，并在文件末尾写数据。如文件不存在，则建立一个空文件；如文件已经存在，则把原文件打开，并保持原内容不变，文件位置指针指向末尾，新写入的数据追加在文件末尾。

➤ 读写二进制文件

- 因为二进制文件是字节流，因此二进制文件在打开读写时不用指定编码，也不存在`readline`，`readlines`读一行或者多行的操作函数。一般二进制文件使用`read`函数读取，使用`write`函数写入。

1. 写二进制文件

如果要把二进制数据`data`写入文件，则使用`write`函数，格式：

文件对象.write(data)

➤ 读写二进制文件

2. 读二进制文件

文件对象.read()

海量性

文件对象.read(n)

- 如果不指定要读取的字符数n，使用read()读，则读到整个文件的内容。
- 如果使用read(n)指定要读取的字符数，要么就按要求读取n个字符；如果要读n个字符，而文件没有那么多字符，那么就读取所有文件内容。

➤ 读写二进制文件例子

```
In [22]: ► def writeFile():  
            fobj=open('abc.txt','wb')  
            fobj.write('Python文件'.encode())  
            fobj.close()  
  
            def readFile():  
                fobj=open('abc.txt','rb')  
                data=fobj.read()  
                print(data.decode())  
                fobj.close()  
  
writeFile()  
readFile()
```

Python文件

➤ 读写二进制文件例子

```
In [43]: ▶ with open('myModule.py', 'rb') as f:  
          data=f.read()  
          print(data.decode())
```

```
def myMin(a, b):  
    c=a  
    if a>b:  
        c=b  
    return c  
def myMax(a, b):  
    c=a  
    if a<b:  
        c=b  
    return c
```

➤ 读写二进制文件

● 修改程序如下：

海星林

```
In [12]: ► with open("test.bt", "wb+") as fp:
            fp.write(bytes("我爱中国".encode("utf-8"))) #转换成使用utf-8编码
            fp.seek(0) #文件指针定位到开头
            b = fp.read().decode("utf-8") #解码方式和编码方式要一致
            print(b)
```

我爱中国

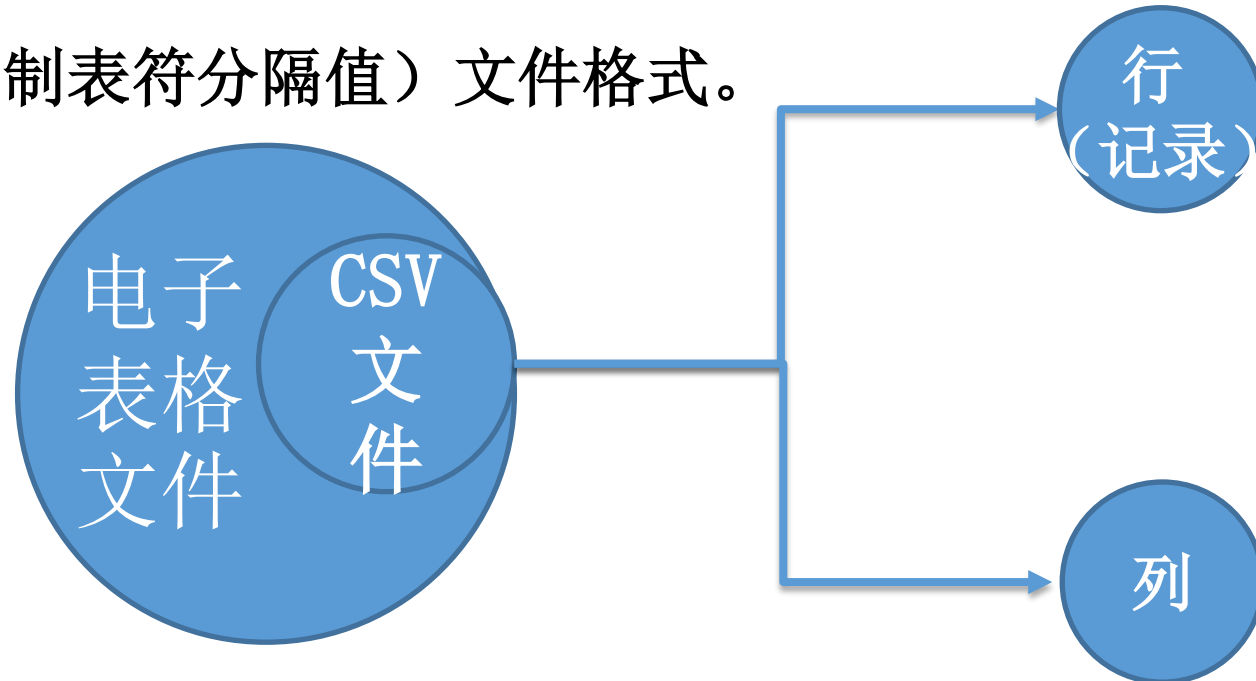
- 可以看出如果直接用文本文件或二进制文件格式存储 Python 中的各种对象，通常需要进行繁琐的转换。

● 7.4.1 CSV文件

- **CSV**格式属于电子表格文件，其中数据存储在单元格内。每个单元格按照行和列结构进行组织。

CSV中的每一行代表一个观察，通常称为一条记录。每个记录可以包含一个或多个由逗号分隔的字段。如果文件中不使用逗号分隔，而是使用制表符进行分隔，这样的文件格式称为**TSV**（制表符分隔值）文件格式。

- **TSV**（制表符分隔值）文件格式。



csv文件的读取和写入

- **csv (comma separated values, 逗号分隔值)**文件是一种用来存储表格数据（数字和文本）的纯文本格式文件，文档的内容是由“,”分隔的一列列的数据构成，它可以被导入各种电子表格和数据库中。纯文本意味着该文件是一个字符序列。
- 在**csv**文件中，数据“栏”(数据所在列，相当于数据库的字段)以逗号分隔，可允许程序通过读取文件为数据重新创建正确的栏结构(如把两个数据栏的数据组合在一起)，并在每次遇到逗号时开始新的一栏。
- **csv**文件由任意数目的记录组成，记录间以某种换行符分隔，一行即为数据表的一行；每条记录由字段组成，字段间的分隔符最常见的是逗号或制表符。
- 可使用**Word**、记事本、**Excel**等方式打开**csv**文件。

csv文件的读取和写入

- 创建csv文件的方法有很多，最常用的方法是用电子表格创建，如Microsoft Excel。在Microsoft Excel中，选择“文件”>“另存为”，然后在“文件类型”下拉选择框中选择 "CSV (逗号分隔)(*.csv)"，然后点击保存即创建了一个csv格式的文件。
- Python的csv模块提供了多种读取和写入csv格式文件的方法。

本节基于consumer.csv文件，其内容为：

客户年龄,平均每次消费金额,平均消费周期

23,318,10

22,147,13

24,172,17

27,194,57

➤ csv文件的读取

csv.reader()用来读取csv文件，其语法格式如下：

csv.reader(csvfile, dialect='excel')

参数说明：

csvfile：可以是文件(file)对象或者列表(list)对象，如果csvfile是文件对象，要求该文件要以newline=""的方式打开，否则两行之间会空一行。

dialect：编码风格，默认为excel的风格，也就是用逗号(,)分隔，dialect方式也支持自定义，通过调用register_dialect方法来注册。

➤ csv文件的读取和写入

csv读取的步骤

0 打开文件

调用open()函数

1 创建对象

借助reader()函数

2 读取内容

遍历reader对象

3 打印内容

print()



► csv文件的读取例子

```
In [14]: ► import csv,shutil, os
os.chdir("C:/2020-2021(2)/bigdata/jupyter code")    # 进入文件所在目录
with open('consumer.csv',newline='') as csvfile:
    spamreader = csv.reader(csvfile)                # 返回的是迭代类型
    for row in spamreader:
        print(','.join(row))                        #以逗号连接各字段
    csvfile.seek(0)                                  #文件指针移动到文件开始
    for row in spamreader:
        print(row)
```

客户年龄 ， 平均每次消费金额 ， 平均消费周期

23, 318, 10

22, 147, 13

24, 172, 17

27, 194, 57

['客户年龄 ', '平均每次消费金额 ', '平均消费周期']

['23', '318', '10']

['22', '147', '13']

['24', '172', '17']

['27', '194', '57']

➤ csv文件的写入

csv.writer()用来写入csv文件，其语法格式如下：

csv.writer(csvfile, dialect='excel', **fmtparams)

说明：**csv.writer()**返回一个**csv.writer**对象，使用**csv.writer**对象可将用户的数据写入该**csv.writer**对象所对应的文件里。

csvfile：可以是文件(file)对象或者列表(list)对象。

dialect：编码风格，默认为excel的风格，也就是用逗号(,)分隔，**dialect**方式也支持自定义，通过调用**register_dialect**方法来注册。

➤ **csv.writer()**所生成的**csv.writer**文件对象支持以下写入csv文件的方法：

writerow(row)：写入一行数据

writerows(rows)：写入多行数据

➤ csv文件的写入

csv写入的步骤

0 创建文件

调用open()函数

1 创建对象

借助writer()函数

2 写入内容

调用writer对象的writerow()方法

3 关闭文件

close()

➤ csv文件的写入

csv.writer()用来写入csv文件，其语法格式如下：

csv.writer(csvfile, dialect='excel', **fmtparams)

说明：**csv.writer()**返回一个**csv.writer**对象，使用**csv.writer**对象可将用户的数据写入该**csv.writer**对象所对应的文件里。

csvfile：可以是文件(file)对象或者列表(list)对象。

dialect：编码风格，默认为excel的风格，也就是用逗号(,)分隔，**dialect**方式也支持自定义，通过调用**register_dialect**方法来注册。

➤ **csv.writer()**所生成的**csv.writer**文件对象支持以下写入csv文件的方法：

writerow(row)：写入一行数据

writerows(rows)：写入多行数据



➤ csv文件的写入例子

```
In [15]: ► import csv,shutil, os
os.chdir("C:/2020-2021(2)/bigdata/jupyter code") # 进入文件所在目录
with open('consumer.csv', 'w', newline='') as csvfile:
    #写入的数据将覆盖consumer.csv文件
    spamwriter = csv.writer(csvfile) #生成csv.writer文件对象
    spamwriter.writerow(['55', '555', '55']) #写入一行数据
    spamwriter.writerows([('35', '355', '35'), ('18', '188', '18')])
with open('consumer.csv', newline='') as csvfile: #重新打开文件
    spamreader = csv.reader(csvfile)
    for row in spamreader: #输出数据文件
        print(row)

['55', '555', '55']
['35', '355', '35']
['18', '188', '18']
```



➤ csv文件的写入例子

```
In [20]: ▶ import csv
def csv_write(path, data):
    with open(path, 'w', encoding='utf-8', newline='') as f:
        writer = csv.writer(f, dialect='excel')
        for row in data:
            writer.writerow(row)
    return True
data = [
    ['姓名', '年龄', '身高 (cm)', '体重 (kg)'],
    ['张三', 38, '176cm', '75'],
    ['李四', 25, '160cm', '46'],
    ['王五', 28, '170cm', '62']
]
csv_write('persons.csv', data)
```

Out[20]: True

➤ 获取数据的重要性

数据是大数据处理的基础，获取数据是数据处理和分析的第一步。

➤ 如何获取数据

（1）从一些开源的网站上直接下载相关数据集，比如国外的kaggle和国内的阿里天池。

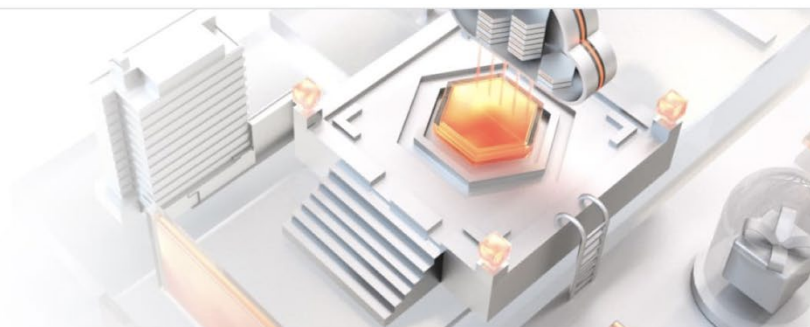
<https://www.kaggle.com/> 数据集，例子程序，大数据处理竞赛，sklearn 中的数据集。

（2）利用相关技术从网页中抓取相关数据，比如爬虫，然后利用获得的数据进行相关数据分析工作。

➤ 阿里天池

Welcome to Tianchi Data Sets

We provide standard datasets for academic use. For information about citing data sets [our citation policy](#)

[+ 新建](#)

行业: [全部](#) [Business](#) [Internet](#) [Finance](#) [Healthcare](#) [Politics and culture](#) [Computer science](#)

展开

技术: [全部](#) [Computer vision](#) [Natural language processing](#) [Classification](#) [Object detection](#) [Speech recognition](#)

展开

来源: [全部](#) [官方数据](#) [公共数据](#) [我的数据](#) [打榜数据集](#) [HOT](#)



精华 ↑ 更新时间 ↑ 热度 ↑

Large-scale SSD Failure Prediction Dataset大型SSD故障预测数据集 [置顶](#)

Large-scale SSD Failure Prediction Dataset is provided by Alibaba Cloud, Alibaba Group. 本数据集由阿里云提供。

👤 天池小T ⌚ 2021-03-18 📄 45 👤 15 👁 840 💬 0

CreativeRanking: A Large-Scale Dataset for Creative Ranking in Disp... [置顶](#)

CreativeRanking dataset is provided by Alimama, Alibaba. 本数据集由阿里妈妈提供。

➤ 爬虫的定义

如果将互联网比作一张大网，爬虫就像是在网上爬行的蜘蛛。把每个页面比作是一个个节点，网络爬虫访问了该页面并获取了信息就相当于爬过了该节点。简单来讲，爬虫就是一段自动抓取互联网信息的程序，从互联网抓取特定的信息。

